

大数据技术与基础

课后作业

学号：2106410128

姓名：焦雨欣

班级：计算机 2101

抓取一个公开网站上的数据，数据必须是多页，存为 csv 格式；如果数据是多个 pdf，将其合并为一个 pdf。程序要尽量自动化，减少人为干预。

代码：

1. 导入库包

```
import requests
```

```
from lxml import etree
```

```
from time import sleep
```

```
import os
```

```
import pandas as pd
```

```
import re
```

```
BOOKS = []
```

```
IMGURLS = []
```

2. 获取网页源代码

```
def get_html(url):
```

```
    headers = {
```

```
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36 Edg/117.0.2045.36'
```

```
    }
```

```
    # 异常处理
```

```
    try:
```

```
        html = requests.get(url, headers=headers)
```

```
        # 声明编码方式
```

```
        html.encoding = html.apparent_encoding
```

```
        # 判断
```

```
        if html.status_code == 200:
```

```
            print('成功获取源代码')
```

```
            # print(html.text)
```

```
    except Exception as e:
```

```
        print('获取源代码失败： %s' % e)
```

```
    # 返回 html
```

```
    return html.text
```

3. 解析网页源代码

```
def parse_html(html):
```

```
    html = etree.HTML(html)
```

```
    # 每个图书信息分别保存在 class="indent" 的 div 下的 table 标签内
```

```
    tables = html.xpath("//div[@class='indent']//table")
```

```
    # print(len(tables)) # 打印之后如果是 25 的话就是对的
```

```
    books = []
```

```

imgUrls = []
# 遍历通过 xpath 得到的 li 标签列表
# 因为要获取标题文本，所以 xpath 表达式要追加 /text(), t.xpath 返回的是一个列表，且列表中只有一个元素所以追加一个[0]
for t in tables:
    # title = t.xpath("//div[@class='p12']/a/@title") # 匹配得到的是空的
    # 书名
    title = t.xpath("//td[@valign='top']/a/@title")[0]
    # 链接
    link = t.xpath("//td[@valign='top']/a/@href")[0]
    # 获取 pl 标签的字符串
    pl = t.xpath("//td[@valign='top']/p[1]/text()")[0]
    # 截取国家
    if '[' in pl: country = pl.split(' ')[1].split(' ')[0]
    else: country = '中' # 没有国家的默认为“中国”
    # 截取作者
    if '[' in pl:
        author = pl.split(' ')[1].split('/')[0].replace(" ", "")
    elif len(pl.split('/')) == 3:
        author = '无'
    elif len(pl.split('/')) == 2:
        author = pl.split('/')[0]
    elif '[' not in pl:
        if len(pl.split('/')) == 4: author = pl.split('/')[-4]
        elif len(pl.split('/')) == 5: author = pl.split('/')[-5]
        elif len(pl.split('/')) == 6: author = pl.split('/')[-6]
    else: author = '无'
    # 截取翻译者
    if len(pl.split('/')) == 3: translator = ''
    elif '[' in pl: if len(pl.split('/')) == 4: translator = pl.split('/')[-3]
        elif len(pl.split('/')) == 5: translator = pl.split('/')[-4]
        elif len(pl.split('/')) == 6: translator = pl.split('/')[-5]
    else:
        translator = ''
    # 截取出版社
    if len(pl.split('/')) == 2: publisher = pl.split('/')[0]
    elif len(pl.split('/')) == 3: publisher = pl.split('/')[0]
    elif '[' in pl: if len(pl.split('/')) == 4: publisher = pl.split('/')[1]
        elif len(pl.split('/')) == 5: publisher = pl.split('/')[2]
        elif len(pl.split('/')) == 6: publisher = pl.split('/')[-3]
        elif len(pl.split('/')) == 7: publisher = pl.split('/')[-4]
    elif '[' not in pl:
        # if len(pl.split('/')) == 3:
        publisher = pl.split('/')[-3]

```

```

        # if len(pl.split('/')) == 6:
        #     publisher = pl.split('/')[-3]
        # elif len(pl.split('/')) == 7:
        #     publisher = pl.split('/')[-4]
        # 截取出版时间
        if len(pl.split('/')) == 2: time = '不详'
        elif len(pl.split('/')) == 4: time = pl.split('/')[-2]
        elif len(pl.split('/')) == 5: time = pl.split('/')[-2]
        elif len(pl.split('/')) == 6: time = pl.split('/')[-2]
        # 截取单价
        if '元' in pl: price = pl.split('/')[-1].split('元')[0]
        else: price = pl.split('/')[-1]
        # 获取星级数
        str1 = t.xpath("//*[td[@valign='top']//div[@class='star
clearfix']/span[1]/@class")[0].replace("allstar", "")
        # 此时获取到的数字其实是字符串类型，不能直接%10，需要把 str 转
        化为 int
        num = int(str1)
        star = num / 10
        # 获取评分
        score = t.xpath("//*[td[@valign='top']//div[@class='star
clearfix']/span[2]/text())[0]
        # 获取评价人数
        pnum = t.xpath("//*[td[@valign='top']//div[@class='star
clearfix']/span[3]/text())[0]
        people = re.sub("\D", "", pnum)
        # 获取简介
        comments = t.xpath("//*[p[@class='quote']/span/text())"
        comment = comments[0] if len(comments) != 0 else "无"
        book = {
            '书名': title, '链接': link, '国家': country, '作者': author, '翻译者':
translator, '出版社': publisher, '出版时间': time, '价格': price, '星级': star, '评分': score,
            '评价人数': people, '简介': comment
        }
        # 图片
        imgUrl = t.xpath("//*[a/img/@src)")[0]
        # print(imgUrl)
        books.append(book)
        imgUrls.append(imgUrl)
    return books, imgUrls
# 4. 下载图片保存文件
def downloading(url, book):
    # 判断文件夹是否在指定路径下面,建立文件夹并把指定路径移到文件夹下
    面

```

```

if 'imgs' in os.listdir(r'D:/pythonProject/111-main'):
    pass
else:
    os.mkdir(r'D:/pythonProject/111-main/imgs')
os.chdir(r'D:/pythonProject/111-main/imgs')
# 返回 img 的二进制流
img = requests.request('GET', url).content

with open(book['书名'] + '.jpg', 'wb') as f:
    # print('正在下载: %s' % url)
    f.write(img)
# 5. 数据预处理
# def processData():

if __name__ == '__main__':
    # url = 'https://book.douban.com/top250?start=0'
    # 10 页循环遍历
    for i in range(4):
        # 2. 定义 url 并获取网页源代码
        url = 'https://book.douban.com/top250?start={}'.format(i * 25)
        # print(url)
        html = get_html(url)
        # 3. 解析网页源代码
        sleep(1)
        books = parse_html(html)[0]
        imgUrls = parse_html(html)[1]
        BOOKS.extend(books)
        IMGURLS.extend(imgUrls)
    # 6. 下载图片保存文件
    for i in range(100):
        # sleep(1)
        downloading(IMGURLS[i], BOOKS[i])
    os.chdir(r'D:/pythonProject/111-main/imgs')
    # 以 csv 格式写入本地
    bookdata = pd.DataFrame(BOOKS)
    bookdata.to_csv('D:/pythonProject/111-main/book.csv', index=False)
    print("图书信息写入本地成功")
    #

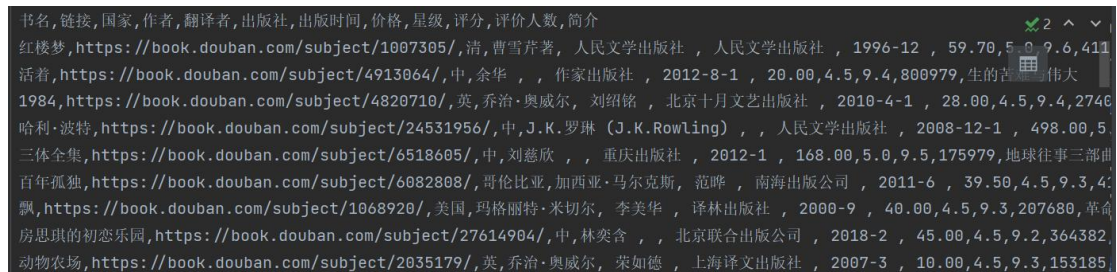
```

程序说明:

使用请求库向指定的 URL 发出 GET 请求。然后从响应中提取数据，并将其保存为 csv 文件。提取的数据为豆瓣网站获取图书信息，包括图

书的书名、链接、作者、国家、翻译者、出版社、出版时间、价格、星级、评分、评价人数

运行截图：



```
书名,链接,国家,作者,翻译者,出版社,出版时间,价格,星级,评分,评价人数,简介
红楼梦,https://book.douban.com/subject/1007305/,清,曹雪芹著,人民文学出版社,人民文学出版社,1996-12,59.70,5.0,9.6,411
活着,https://book.douban.com/subject/4913064/,中,余华,,作家出版社,2012-8-1,20.00,4.5,9.4,800979,生的苦,死的大
1984,https://book.douban.com/subject/4820710/,英,乔治·奥威尔,刘绍铭,北京十月文艺出版社,2010-4-1,28.00,4.5,9.4,2740
哈利·波特,https://book.douban.com/subject/24531956/,中,J.K.罗琳(J.K.Rowling),人民文学出版社,2008-12-1,498.00,5.0,9.8,1000000
三体全集,https://book.douban.com/subject/6518605/,中,刘慈欣,,重庆出版社,2012-1,168.00,5.0,9.5,175979,地球往事三部曲
百年孤独,https://book.douban.com/subject/6082808/,哥伦比亚,加西亚·马尔克斯,范晔,南海出版公司,2011-6,39.50,4.5,9.3,41
飘,https://book.douban.com/subject/1068920/,美国,玛格丽特·米切尔,李美华,译林出版社,2000-9,40.00,4.5,9.3,207680,革命
房思琪的初恋乐园,https://book.douban.com/subject/27614904/,中,林奕华,,北京联合出版公司,2018-2,45.00,4.5,9.2,364382
动物农场,https://book.douban.com/subject/2035179/,英,乔治·奥威尔,荣如德,上海译文出版社,2007-3,10.00,4.5,9.3,153185
```

葡萄酒数据集(wine.data)搜集了法国不同产区葡萄酒的化学指标。试建立决策树、随机森林和 xgBoost3 种分类器模型，比较各种分类器在此数据集上的效果。

【提示】：每种分类器，需要对参数进行尝试，找出此种分类算法的较优模型，再与其他分类器性能进行比较。

代码：

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# 读取数据集
data = pd.read_csv('wine.data', header=None)

# 将数据集分为特征和标签
X = data.iloc[:, 1:]
y = data.iloc[:, 0]
from sklearn.preprocessing import LabelEncoder
#XGBoost 要求目标变量必须是从 0 开始的整数，而 y 变量取值是 [1, 2, 3],使用 LabelEncoder 进行转换
le = LabelEncoder()
xgy = le.fit_transform(y)
```

```

# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy, test_size=0.3,
random_state=42)
# 建立决策树模型
dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)
dtc_score = dtc.score(X_test, y_test)
# 建立随机森林模型
rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
rfc_score = rfc.score(X_test, y_test)
# 建立 xgBoost 模型
xgb = XGBClassifier(random_state=42)
xgb.fit(X_train, xgy_train)
xgb_score = xgb.score(X_test, xgy_test)
# 输出各个模型在测试集上的表现
print('Decision Tree Classifier score:', dtc_score)
print('Random Forest Classifier score:', rfc_score)
print('XGBoost Classifier score:', xgb_score)

```

程序说明：

从 wine.data 中读取数据将数据集中除第一列外的所有列赋值给变量 X，将数据集中第一列赋值给变量 y。使用 LabelEncoder 进行对 y 的值进行转换，将数据集分为训练集和测试集，test_size 为 0.3, random_state 为 42, 分别建立三种模型，并进行测试，输出各个模型在测试集上的表现，得出随机森林的模型最好

运行截图

```

Decision Tree Classifier score: 0.9629629629629629
Random Forest Classifier score: 1.0
XGBoost Classifier score: 0.9629629629629629

```